4

MÉTODOS DE MONTE CARLO

INTRODUCCIÓN

- Supongamos que un entorno en donde cada celda es un estado.
- También supongamos que un agente interactúa con ese entorno y recibe una recompensa a medida que se desplaza aleatoriamente hacia su objetivo.

INTRODUCCIÓN

- Si el valor de cada estado v(s) se calculara promediando las recompensas que recibe el agente en diferentes episodios y su secuencia de acciones actual fuese "Abajo, Abajo, Abajo, Derecha" tendríamos (al final del episodio 1):

v(N) = 0 (porque es un estado absorbente).

v(M) = -5 + 0 = -5

v(I) = -0.1 + -5 = -5.1

...

INTRODUCCIÓN

- Para el episodio 2 tendríamos (si la secuencia fuese "Abajo, Abajo, Derecha, Abajo"):

v(N) = 0 (porque es un estado absorbente).

v(J) = -5 + 0 = -5

"v(I)" = -0.1 + -5 = -5.1

v(I) = (-5.1 + -5.1) / 2 = -5.1

...

INTRODUCCIÓN

- Las funciones de valor v(s) y q(s, a) que vimos hasta ahora se pueden estimar usando un enfoque distinto.

- En vez de usar la ecuación de Bellman podemos promediar las recompensas de cada episodio y tomar ese promedio como el valor v(s) o q(s, a).

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

- Veamos esto con un ejemplo un poco mas completo:
- Supongamos un buscador de tesoro en un laberinto de 3x3.

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

- Estados: {A, B, C, D, E, F, G, H, I}
- Estado Inicial: Siempre empieza en A.
- Estados Terminales:
  - ✓ I (Tesoro): Si llega a I, el episodio termina y recibe una recompensa grande (+10).
  - ✓ F (Trampa): Si llega a F, el episodio termina y recibe una penalización (-5)
- Política (π) Fija (desde cualquier casilla no terminal):
  - ✓ 60% de probabilidad de intentar moverse a la DERECHA.
  - ✓ 40% de probabilidad de intentar moverse ABAJO.
  - ✓ Si el movimiento elegido lo saca del laberinto (choca con un muro imaginario), se queda en la misma casilla para ese paso.
- Recompensas (R):
  - ✓ Entrar en I: +10
  - ✓ Entrar en F: -5
  - ✓ Cualquier otro movimiento (o quedarse quieto por chocar): -0.1 (un pequeño costo por cada paso/tiempo).
- Factor de Descuento: $\gamma = 1$ (sin descuento, para simplificar). El valor es la suma total de recompensas futuras.

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

- Inicialización:
  - ✓ V(s) = 0 para todos los estados s ∈ {A, B, C, D, E, G, H}. (F e I son terminales).
  - ✓ Returns(s) = [] para todos los estados s ∈ {A, B, C, D, E, G, H}.

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

- Episodio 1:
- Empieza en A. La secuencia de movimientos aleatorios según la política (π) es: Derecha, Abajo, Derecha.
  - ✓ A -> B (R = -0.1)
  - ✓ B -> E (R = -0.1)
  - ✓ E -> F (Trampa) (R = -5) --> Episodio Termina.
- Secuencia: A, B(-0.1), E(-0.1), F(-5)

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

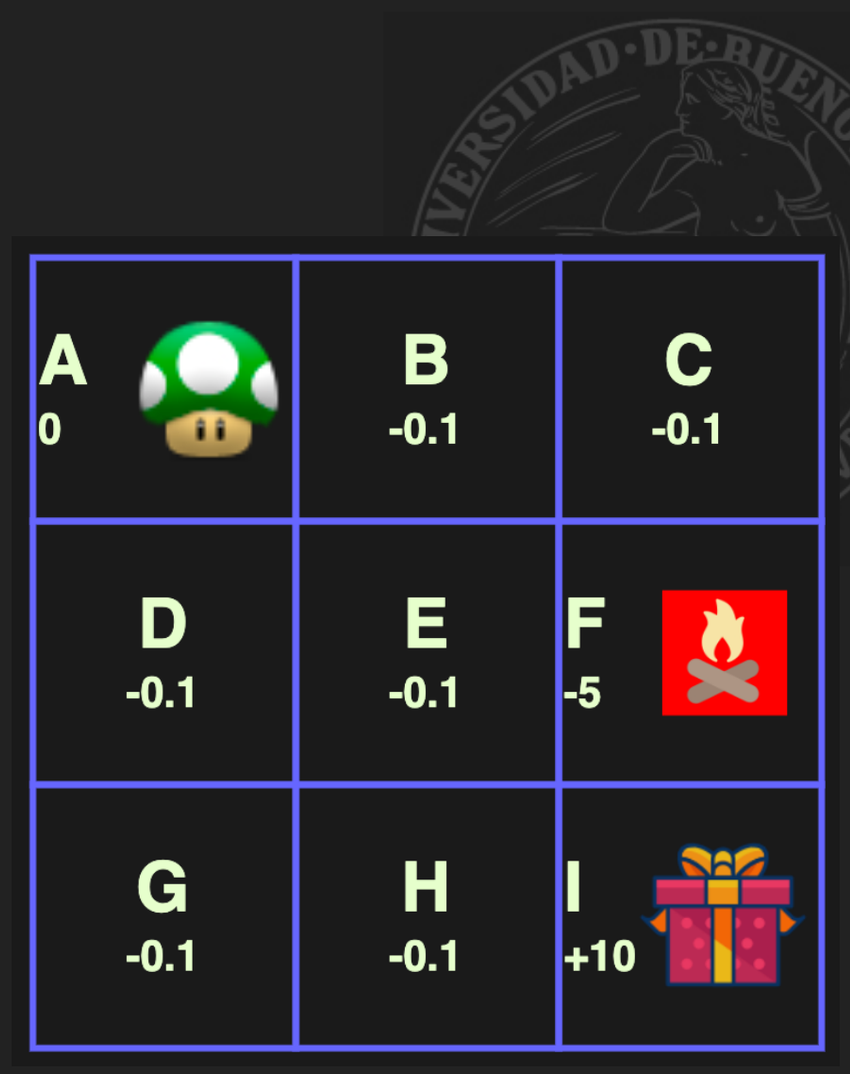
- Cálculo de Retornos (G) Hacia Atrás ($\gamma=1$):
  - ✓ G en F (terminal) = 0 (por definición)
  - ✓ G en E (justo antes de F): G = R(E->F) + G_en_F = -5 + 0 = -5
  - ✓ G en B (justo antes de E): G = R(B->E) + G_en_E = -0.1 + (-5) = -5.1
  - ✓ G en A (justo antes de B): G = R(A->B) + G_en_B = -0.1 + (-5.1) = -5.2

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

- Actualización de Valores (Solo Primera Visita):
  - ✓ Estado E: ¿Primera visita en este episodio? Sí. Returns(E) = [-5]. V(E) = -5 / 1 = -5.
  - ✓ Estado B: ¿Primera visita en este episodio? Sí. Returns(B) = [-5.1]. V(B) = -5.1 / 1 = -5.1.
  - ✓ Estado A: ¿Primera visita en este episodio? Sí. Returns(A) = [-5.2]. V(A) = -5.2 / 1 = -5.2.
- Valores después Ep. 1: V(A)=-5.2, V(B)=-5.1, V(E)=-5, otros=0

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

- Episodio 2:
- Empieza en A. Secuencia aleatoria según π: Abajo, Derecha, Derecha.
  - ✓ A -> D (R = -0.1)
  - ✓ D -> E (R = -0.1)
  - ✓ E -> F (Trampa) (R = -5) --> Episodio Termina.
- Secuencia: A, D(-0.1), E(-0.1), F(-5)



MONTE CARLO DE PRIMERA VISITA – EJEMPLO

- Cálculo de Retornos (G) Hacia Atrás (γ=1):
  - ✓ G en F = 0
  - ✓ G en E = R(E->F) + G_en_F = -5 + 0 = -5
  - ✓ G en D = R(D->E) + G_en_E = -0.1 + (-5) = -5.1
  - ✓ G en A = R(A->D) + G_en_D = -0.1 + (-5.1) = -5.2

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

- Actualización de Valores (Solo Primera Visita):
  - ✓ Estado E: ¿Primera visita? Sí. Returns(E) = [-5, -5]. V(E) = (-5 + -5) / 2 = -5.
  - ✓ Estado D: ¿Primera visita? Sí. Returns(D) = [-5.1]. V(D) = -5.1 / 1 = -5.1.
  - ✓ Estado A: ¿Primera visita? Sí. Returns(A) = [-5.2, -5.2]. V(A) = (-5.2 + -5.2) / 2 = -5.2.
- Valores después Ep. 2: V(A)=-5.2, V(B)=-5.1, V(D)=-5.1, V(E)=-5, otros=0

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

- Episodio 3:
- Empieza en A. Secuencia aleatoria según π: Derecha, Derecha, Abajo, Abajo.
  - ✓ A -> B (R = -0.1)
  - ✓ B -> C (R = -0.1)
  - ✓ C -> F (Trampa) (R = -5) --> Episodio Termina.
- Secuencia: A, B(-0.1), C(-0.1), F(-5)

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

- Cálculo de Retornos (G): G en C=-5, G en B=-5.1, G en A=-5.2.
- Actualización de Valores (Solo Primera Visita):
  - ✓ Estado C: ¿Primera visita? Sí. Returns(C) = [-5]. V(C) = -5.
  - ✓ Estado B: ¿Primera visita? Sí. Returns(B) = [-5.1, -5.1]. V(B) = -5.1.
  - ✓ Estado A: ¿Primera visita? Sí. Returns(A) = [-5.2, -5.2, -5.2]. V(A) = -5.2.
- Valores después Ep. 3: V(A)=-5.2, V(B)=-5.1, V(C)=-5, V(D)=-5.1, V(E)=-5, otros=0

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

- Episodio 4:
- Empieza en A. Secuencia aleatoria según π: Abajo, Derecha, Abajo, Derecha.
  - ✓ A -> D (R = -0.1)
  - ✓ D -> E (R = -0.1)
  - ✓ E -> H (R = -0.1)
  - ✓ H -> I (Tesoro) (R = +10) --> Episodio Termina.
- Secuencia: A, D(-0.1), E(-0.1), H(-0.1), I(+10)

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

- Cálculo de Retornos (G):
  - ✓ G en I = 0
  - ✓ G en H = R(H->I) + G_en_I = 10 + 0 = 10
  - ✓ G en E = R(E->H) + G_en_H = -0.1 + 10 = 9.9
  - ✓ G en D = R(D->E) + G_en_E = -0.1 + 9.9 = 9.8
  - ✓ G en A = R(A->D) + G_en_D = -0.1 + 9.8 = 9.7

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

- Actualización de Valores (Solo Primera Visita):
  - ✓ Estado H: ¿Primera visita? Sí. Returns(H) = [10]. V(H) = 10.
  - ✓ Estado E: ¿Primera visita? Sí. Returns(E) = [-5, -5, 9.9]. V(E) = (-5 -5 + 9.9) / 3 ≈ -0.03.
  - ✓ Estado D: ¿Primera visita? Sí. Returns(D) = [-5.1, 9.8]. V(D) = (-5.1 + 9.8) / 2 = 2.35.
  - ✓ Estado A: ¿Primera visita? Sí. Returns(A) = [-5.2, -5.2, -5.2, 9.7]. V(A) = (-5.2*3 + 9.7) / 4 = -1.475.
- Valores después Ep. 4: V(A)≈-1.48, V(B)=-5.1, V(C)=-5, V(D)=2.35, V(E)≈-0.03, V(H)=10, otros=0

MONTE CARLO DE PRIMERA VISITA – EJEMPLO

- Se continua generando episodios. Cada vez que se visita un estado por primera vez en un episodio, se añade su retorno calculado a la lista y se recalcula el valor promedio para ese estado.
- Si el agente, por ejemplo, en un episodio hiciera A -> B -> E -> B -> C -> F, la segunda visita a B no se usaría para actualizar V(B). Solo se usaría el retorno calculado desde la primera vez que llegó a B en ese episodio.
- Con suficientes episodios, los valores V(s) convergerán a una estimación de lo bueno que es estar en cada casilla, dada la política de movimiento aleatoria y fija del agente.
- Sería posible ver que estados como H (cerca del tesoro) tienden a tener valores altos, mientras que estados como C (cerca de la trampa) tienden a tener valores bajos.

MONTE CARLO DE PRIMERA VISITA – PSEUDOCODIGO

First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated
Initialize:
    $V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$
    $Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):
    Generate an episode following π: $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$
    $G \leftarrow 0$
    Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
        $G \leftarrow \gamma G + R_{t+1}$
        Unless S_t appears in $S_0, S_1, \ldots, S_{t-1}$:
            Append G to $Returns(S_t)$
            $V(S_t) \leftarrow \text{average}(Returns(S_t))$

MONTE CARLO – CONCEPTO

- El núcleo de los métodos de Monte Carlo es aprender simplemente promediando las recompensas que se obtienen durante muchos intentos, a los que llamamos "episodios".
- Se interactúa con un entorno.
- El agente podría ser un robot real moviéndose por una habitación o tal vez una computadora jugando a un juego.
- El punto es que solo necesita esa interacción, no necesita que alguien le diga las reglas del juego. Lo descubre por sí solo.

MONTE CARLO - CONCEPTO

- El núcleo de los métodos de Monte Carlo es aprender simplemente promediando las recompensas que se obtienen durante muchos intentos, a los que llamamos "episodios".
- El agente que interactúa con su entorno podría ser un robot real moviéndose por una habitación o tal vez una computadora jugando a un juego.
- El punto es que solo necesita esa interacción, no necesita que alguien le diga las reglas del juego. Lo descubre por sí solo.
- Al igual que cuando usábamos los algoritmos de Iteración de Política e Iteración de Valor, con MC lo que se busca es averiguar lo que se llama la función de valor de estado.

MONTE CARLO – CONCEPTO

- Los métodos de Monte Carlo funcionan mediante muestreo aleatorio para obtener respuestas numéricas.
- Fueron desarrollados originalmente en la década de 1940 para tratar problemas matemáticos complejos que carecían de soluciones analíticas.
- Su nombre, “Monte Carlo", se eligió por la similitud entre este proceso basado en la aleatoriedad y los juegos de azar del famoso casino.

MONTE CARLO - CONCEPTO

- Existen dos enfoques principales de Monte Carlo para encontrar la función de valor de estado v(s) :

  ✓ MC de primera visita (first-visit MC) y,

  ✓ MC de cada visita (every-visit MC).

MONTE CARLO – CONCEPTO

- En MC de primera visita, para cada ejecución (cada episodio), por cada estado que ve, solo importa lo que sucede después de la primera vez que ve ese estado en ese episodio.
- De modo que se promedian las recompensas futuras de esa primera visita a través de todos los diferentes episodios.
- Si el agente se encuentra con ese mismo estado más tarde en ese mismo episodio, simplemente lo ignora. Solo se enfoca en esa primera visita.
- El método de MC de cada visita, usa todas las visitas. Cada vez que un estado aparece en un episodio, registra la recompensa desde ese punto en adelante y la agrega al promedio para ese estado.
- El método de MC de primera visita es el mas estudiado, eso es porque tiene algunas características interesantes, especialmente en lo que respecta a las estadísticas.
- Las muestras que se promedian son independientes, por lo que la ley de los grandes números entra en juego. Lo que eso significa es que el valor estimado está garantizado a acercarse cada vez más al valor verdadero real cuanto más se visite ese estado. Más visitas, mejor precisión.

MONTE CARLO DE CADA VISITA – EJEMPLO

- Estados: {A, B, C, D, E, F, G, H, I}
- Estado Inicial: Siempre empezamos en A.
- Estados Terminales: I (+10 recompensa al entrar), F (-5 recompensa al entrar).
- Nueva Política (π) Fija: Desde cualquier casilla no terminal, el agente elige una de las cuatro direcciones (Arriba, Abajo, Izquierda, Derecha) con igual probabilidad (25% cada una). Si el movimiento elegido choca contra un muro (fuera del 3x3), el agente se queda en la misma casilla para ese paso.
- Recompensas (R): +10 (entrar I), -5 (entrar F), -0.1 (cualquier otro movimiento o quedarse quieto).
- Factor de Descuento (γ): 1.

MONTE CARLO DE CADA VISITA – EJEMPLO

- Inicialización:
  - ✓ V(s) = 0 para s ∈ {A, B, C, D, E, G, H}.
  - ✓ Returns(s) = [] para s ∈ {A, B, C, D, E, G, H}.

MONTE CARLO DE CADA VISITA – EJEMPLO

- Episodio 1: (Misma secuencia que antes, ahora es válida bajo la nueva política)
- Supongamos que la secuencia de movimientos aleatorios es: Derecha, Abajo, Derecha.
- Secuencia: A, B(-0.1), E(-0.1), F(-5) --> Termina

MONTE CARLO DE CADA VISITA – EJEMPLO

- Cálculo de Retornos (G) Hacia Atrás (γ=1):
  - ✓ G en F = 0
  - ✓ G en E = -5
  - ✓ G en B = -5.1
  - ✓ G en A = -5.2

MONTE CARLO DE CADA VISITA – EJEMPLO

- Actualización de Valores (Cada Visita): Recorremos (A, B, E).
  - ✓ Visita E (t=2): Añadir -5 a Returns(E). R(E)=[-5]. V(E) = -5.
  - ✓ Visita B (t=1): Añadir -5.1 a Returns(B). R(B)=[-5.1]. V(B) = -5.1.
  - ✓ Visita A (t=0): Añadir -5.2 a Returns(A). R(A)=[-5.2]. V(A) = -5.2.
- Valores después Ep. 1: V(A)=-5.2, V(B)=-5.1, V(E)=-5, otros=0.

MONTE CARLO DE CADA VISITA – EJEMPLO

- Episodio 2: (Misma secuencia que antes)
- Supongamos secuencia: Abajo, Derecha, Derecha.
- Secuencia: A, D(-0.1), E(-0.1), F(-5) --> Termina

MONTE CARLO DE CADA VISITA – EJEMPLO

- Cálculo de Retornos (G):
  - ✓ G en F = 0, G en E = -5, G en D = -5.1, G en A = -5.2
- Actualización de Valores (Cada Visita): Recorremos (A, D, E).
  - ✓ Visita E (t=2): Añadir -5 a R(E). R(E)=[-5, -5]. V(E) = (-5-5)/2 = -5.
  - ✓ Visita D (t=1): Añadir -5.1 a R(D). R(D)=[-5.1]. V(D) = -5.1.
  - ✓ Visita A (t=0): Añadir -5.2 a R(A). R(A)=[-5.2, -5.2]. V(A) = (-5.2-5.2)/2 = -5.2.
- Valores después Ep. 2: V(A)=-5.2, V(B)=-5.1, V(D)=-5.1, V(E)=-5, otros=0.

MONTE CARLO DE CADA VISITA – EJEMPLO

- Episodio 3 (revisita):
- Supongamos secuencia: Derecha, Abajo, Arriba, Derecha, Derecha. (E -> B es ahora un movimiento válido)
- Secuencia: A, B(-0.1), E(-0.1), B(-0.1), C(-0.1), F(-5) --> Termina

MONTE CARLO DE CADA VISITA – EJEMPLO

- Cálculo de Retornos (G):
  - ✓ G en F = 0
  - ✓ G en C (antes de F): -5 + 0 = -5
  - ✓ G en B (antes de C, segunda visita a B, t=3): -0.1 + (-5) = -5.1
  - ✓ G en E (antes de B, t=2): -0.1 + (-5.1) = -5.2
  - ✓ G en B (antes de E, primera visita a B, t=1): -0.1 + (-5.2) = -5.3
  - ✓ G en A (antes de B, t=0): -0.1 + (-5.3) = -5.4

MONTE CARLO DE CADA VISITA – EJEMPLO

- Actualización de Valores (Cada Visita): Recorremos (A, B, E, B, C).
  - ✓ Estado Actual (antes Ep 3): V(A)=-5.2, V(B)=-5.1, V(D)=-5.1, V(E)=-5, V(C)=0, V(H)=0

- Actualizaciones:
  - ✓ Visita C (t=4): Añadir G=-5 a R(C). R(C)=[-5]. V(C) = -5.
  - ✓ Visita B (t=3, 2da visita): Añadir G=-5.1 a R(B). R(B) era [-5.1], ahora [-5.1, -5.1]. V(B) = Promedio([-5.1, -5.1]) = -5.1.
  - ✓ Visita E (t=2): Añadir G=-5.2 a R(E). R(E) era [-5, -5], ahora [-5, -5, -5.2]. V(E) = Promedio([-5, -5, -5.2]) = -15.2 / 3 ≈ -5.07.
  - ✓ Visita B (t=1, 1ra visita): Añadir G=-5.3 a R(B). R(B) era [-5.1, -5.1], ahora [-5.1, -5.1, -5.3]. V(B) = Promedio([-5.1, -5.1, -5.3]) = -15.5 / 3 ≈ -5.17.
  - ✓ Visita A (t=0): Añadir G=-5.4 a R(A). R(A) era [-5.2, -5.2], ahora [-5.2, -5.2, -5.4]. V(A) = Promedio([-5.2, -5.2, -5.4]) = -15.8 / 3 ≈ -5.27.
- Valores después Ep. 3: V(A)≈-5.27, V(B)≈-5.17, V(C)=-5, V(D)=-5.1, V(E)≈-5.07, V(H)=0. (La revisita a B afectó los promedios de B, E y A).

MONTE CARLO DE CADA VISITA – EJEMPLO

- Episodio 4: (Misma secuencia que antes, llegando al tesoro)
- Supongamos secuencia: Abajo, Derecha, Abajo, Derecha.
- Secuencia: A, D(-0.1), E(-0.1), H(-0.1), I(+10) --> Termina

MONTE CARLO DE CADA VISITA – EJEMPLO

- Cálculo de Retornos (G):
  - ✓ G en I = 0
  - ✓ G en H = 10
  - ✓ G en E = 9.9
  - ✓ G en D = 9.8
  - ✓ G en A = 9.7

MONTE CARLO DE CADA VISITA – EJEMPLO

- Actualización de Valores (Cada Visita): Recorremos (A, D, E, H).
  - ✓ Estado Actual (antes Ep 4): V(A)≈-5.27, V(B)≈-5.17, V(C)=-5, V(D)=-5.1, V(E)≈-5.07, V(H)=0
- Actualizaciones:
  - ✓ Visita H (t=3): Añadir G=10 a R(H). R(H)=[10]. V(H) = 10.
  - ✓ Visita E (t=2): Añadir G=9.9 a R(E). R(E) era [-5, -5, -5.2], ahora [-5, -5, -5.2, 9.9]. V(E) = Promedio([-5, -5, -5.2, 9.9]) = -5.3 / 4 = -1.325.
  - ✓ Visita D (t=1): Añadir G=9.8 a R(D). R(D) era [-5.1], ahora [-5.1, 9.8]. V(D) = Promedio([-5.1, 9.8]) = 4.7 / 2 = 2.35.
  - ✓ Visita A (t=0): Añadir G=9.7 a R(A). R(A) era [-5.2, -5.2, -5.4], ahora [-5.2, -5.2, -5.4, 9.7]. V(A) = Promedio([-5.2, -5.2, -5.4, 9.7]) = -6.1 / 4 = -1.525.

MONTE CARLO DE CADA VISITA – EJEMPLO

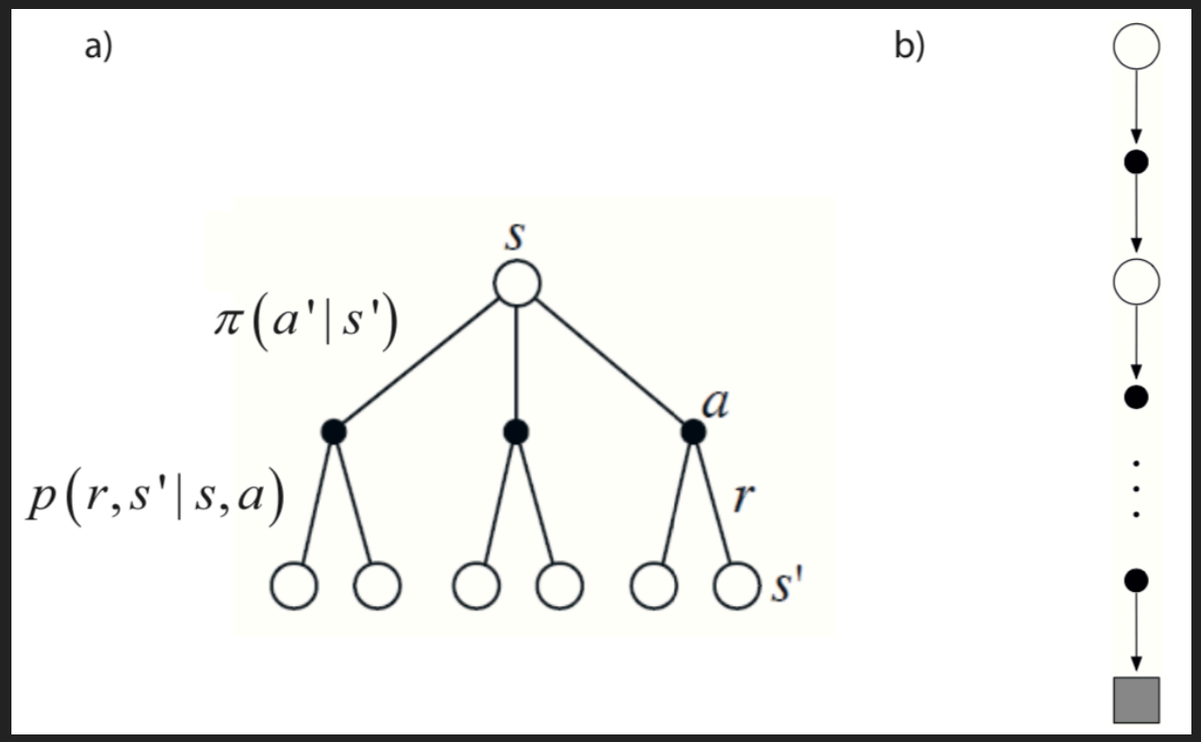
- Valores Finales después de 4 Episodios:
  - ✓ V(A) ≈ -1.53
  - ✓ V(B) ≈ -5.17
  - ✓ V(C) = -5
  - ✓ V(D) = 2.35
  - ✓ V(E) ≈ -1.33
  - ✓ V(G) = 0 (No visitado)
  - ✓ V(H) = 10

MONTE CARLO DE CADA VISITA – EJEMPLO

- Al permitir el movimiento hacia arriba, la secuencia del Episodio 3 que contenía la revisita (E -> B) se vuelve válida y natural bajo la política.
- El algoritmo MC de Cada Visita procesa esta secuencia utilizando ambas visitas a B para actualizar V(B), V(E) y V(A) con los retornos calculados desde cada uno de esos puntos.
- Después de 4 episodios, empezamos a ver cómo los valores reflejan la experiencia: H (cercano al tesoro) tiene un valor alto, C (cercano a la trampa) tiene un valor bajo, y los estados intermedios como D y E empiezan a mostrar valores positivos debido a la posibilidad de alcanzar el tesoro. Aún se necesitan muchos más episodios para que los valores converjan de manera fiable.

DIFERENCIAS ENTRE MC Y PD

- Los diagramas de respaldo de PD (Programación Dinámica) mostraban cada posible siguiente paso que se podía dar desde un estado, junto con las probabilidades de que ocurriera cada uno y el valor de cada estado resultante.
- El diagrama de respaldo de Monte Carlo es diferente. Muestra la trayectoria real, la secuencia específica de estados, acciones y recompensas que realmente ocurren en un episodio simulado.
- En lugar de mostrar cada posible siguiente paso, solo muestra la única trayectoria que se tomó.
- Es como mostrar la ruta real en un mapa en lugar de todas las posibles rutas que podría haberse tomado.



DIFERENCIAS ENTRE MC Y PD

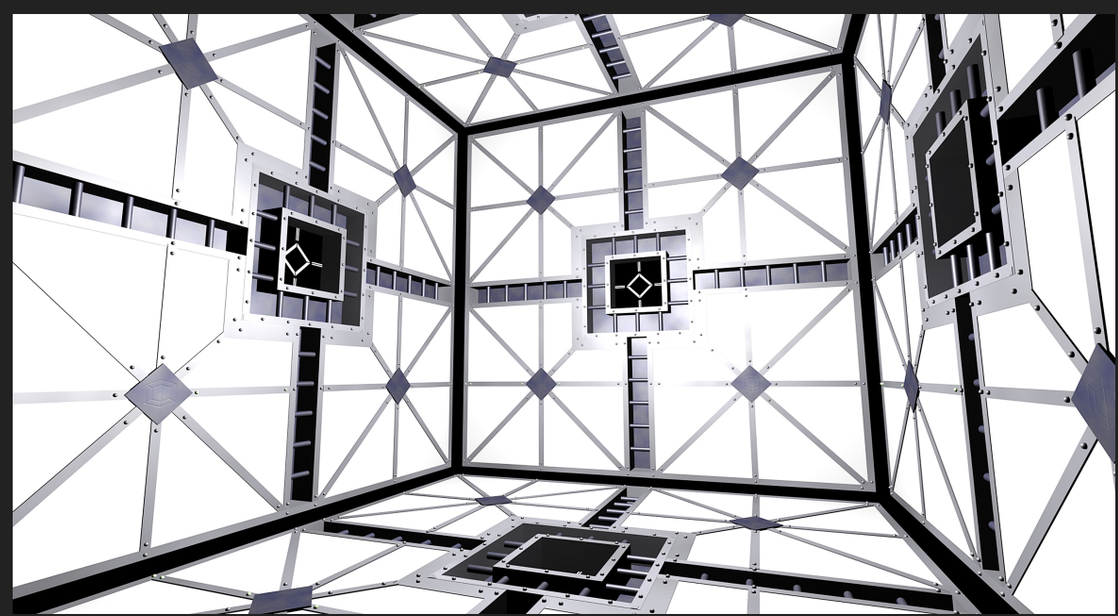
- Monte Carlo no hace “bootstrap".
- El 'bootstrapping' en este contexto significa usar las estimaciones actuales para otros estados para ayudar a estimar el valor del estado que se está analizando.

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r\,|\,s,a)\,[r + \gamma v_\pi(s')]\,,$$

- La programación dinámica hace exactamente eso, es decir que constantemente actualiza los valores basándose en lo que cree que vale el siguiente estado.
- Monte Carlo no hace eso, solo mira las recompensas reales que obtuvo después de estar en ese estado. No se basa en ninguna suposición sobre otros estados.
- Es una forma más directa de averiguar qué tan bueno es un estado, simplemente basándose en lo que realmente sucedió después de estar allí.
- Se ejecutan una gran cantidad de simulaciones que pasen por esos estados y no tiene en cuenta el resto.

ESTIMACIÓN DE LOS VALORES DE ACCIÓN Q(S, A)

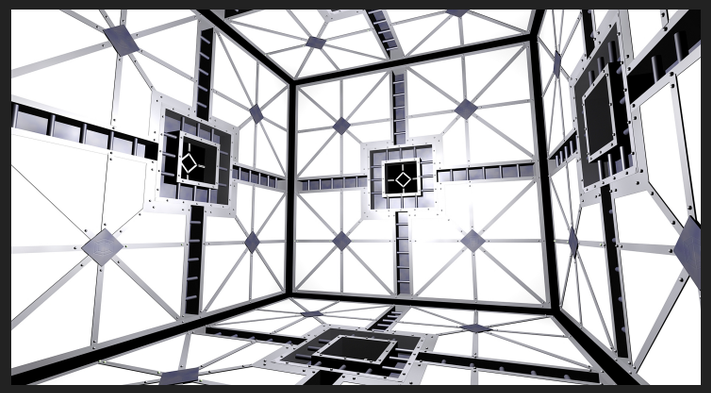
- Si no se tiene un modelo, significa que no se sabe qué va a pasar a continuación o qué recompensas se obtendrá.
- Saber qué tan bueno es un estado no es suficiente. También se necesita saber qué tan bueno es tomar acciones específicas en ese estado, es decir, conocer los valores de acción q(s, a) y luego seguir la política desde allí.
- Analogía: Es como saber lo bueno que es estar en un cierto lugar versus saber lo bueno que es abrir una puerta específica en ese lugar.
- Lo que realmente se busca es averiguar la mejor acción a tomar en cada posible situación. Eso se llama la función de valor de acción óptima.



Fuente: The Cube (1997)

ESTIMACIÓN DE LOS VALORES DE ACCIÓN Q(S, A)

- ¿cómo usar Monte Carlo para estimar estos valores de acción?
- En lugar de solo considerar estados, se consideran pares estado-acción. Esto es, ¿se visitó un estado y luego se tomó una acción en particular? Ese es un par estado-acción.
- Para esto se pueden usar los mismos enfoques, MC de primera visita y de cada visita, para estimar el valor de estos pares. Luego se promedian las recompensas que se obtienen después de tomar esas acciones.
- Analogía: Es como entrar en una habitación y luego abrir una puerta específica, se busca saber, en promedio, qué tan buen resultado se obtiene después de hacer eso.
- En este punto surge un problema. ¿Qué pasa si nunca se prueban ciertas acciones en ciertos estados, especialmente si la política siempre dice que haga lo mismo?
- Si nunca se explora se podría perder obtener un resultado mejor. Es análogo a que si siempre se abre la misma puerta en una habitación, con lo cual podría nunca encontrar el mejor lugar.
- Se necesita probar alternativas diferentes para descubrir qué es realmente lo mejor. Para abordar esto lo más lógico es hacer: Comienzos Exploratorios (Exploring Starts).



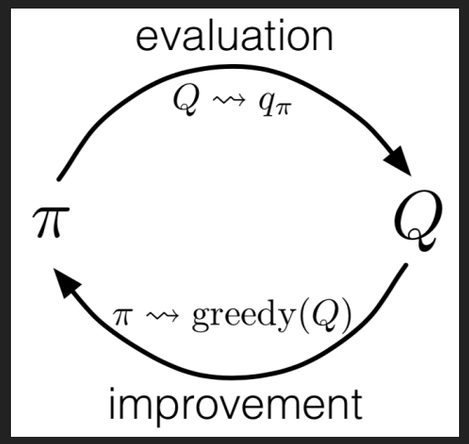
Fuente: The Cube (1997)

COMIENZOS EXPLORATORIOS (EXPLORING STARTS)

- Significa que se asume que cada episodio comienza en un par estado-acción aleatorio.
- Cada posible punto de partida es igualmente probable.
- Esto asegura que, eventualmente, probaremos cada acción en cada estado.
- Es una forma de garantizar que estamos explorando todo.

CONTROL MONTE CARLO

- ¿Cómo pasamos de simplemente averiguar qué tan bueno es un estado a encontrar realmente la mejor manera de actuar?
- Se usa la idea de Iteración de Política Generalizada (GPI), es decir, el ciclo de evaluar la estrategia actual y luego mejorarla basándose en lo que se aprende.
- En PD se usaba el conocimiento del entorno para averiguar qué tan buena era la política, luego se implementaba una política "greedy" (voraz o codiciosa), lo que significa que siempre elegía la acción que parecía ser mejor.
- Con el Control Monte Carlo se aprende qué tan buenas son las acciones probándolas realmente y viendo qué sucede. Luego se mejora la política eligiendo las acciones que parecen llevar a los mejores resultados y para conseguir esto se usa Comienzos Exploratorios.



CONTROL MONTE CARLO

- Monte Carlo ES realiza lo siguiente:
- Comienza eligiendo aleatoriamente un estado y acción de inicio, asegurándose de que cada posible punto de partida tenga una oportunidad, esa es la parte de Comienzos Exploratorios.
- Luego deja que el agente juegue un episodio completo siguiendo su política actual.
- Una vez que termina el episodio, revisa cada paso, y para cada par estado-acción, calcula la recompensa total que obtuvo desde ese punto en adelante. Usa esto para actualizar su estimación de cuán buena es esa acción en ese estado.
- Finalmente, actualiza la política misma, haciendo más probable que elija las acciones que parecen ser las mejores hasta ahora.

Monte Carlo ES (Exploring Starts), for estimating $\pi \approx \pi_*$

Initialize:
  $\pi(s) \in \mathcal{A}(s)$ (arbitrarily), for all $s \in \mathcal{S}$
  $Q(s,a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$
  $Returns(s,a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Loop forever (for each episode):
  Choose $S_0 \in \mathcal{S}$, $A_0 \in \mathcal{A}(S_0)$ randomly such that all pairs have probability > 0
  Generate an episode from S_0, A_0, following π: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
  $G \leftarrow 0$
  Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
    $G \leftarrow \gamma G + R_{t+1}$
    Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1 \ldots, S_{t-1}, A_{t-1}$:
      Append G to $Returns(S_t, A_t)$
      $Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$
      $\pi(S_t) \leftarrow \text{argmax}_a\, Q(S_t, a)$

CONTROL MONTE CARLO – RESUMEN

- Objetivo: Encontrar la política óptima (la mejor forma de moverse), no solo evaluar una fija.
- Valores de Acción (Q): MC ES utiliza Q(s, a), el valor esperado de tomar la acción 'a' en el estado 's' y luego seguir la política. Se necesitamos esto para decidir la mejor acción sin conocer el modelo.
- Exploring Starts (ES): Cada episodio DEBE comenzar en un par estado-acción (s, a) elegido al azar. Esto asegura que eventualmente se prueben todas las acciones en todos los estados.
- Iteración Política Generalizada (GPI): Se alterna entre:
  - ✓ Evaluar Q(s, a) basado en los episodios generados (usando la política actual).
  - ✓ Mejorar la política haciéndola greedy (codiciosa) respecto a los Q(s, a) actuales.
- First-Visit (implícito): Generalmente, para un par (s, a) dado, solo se usa el retorno de la primera vez que aparece en un episodio para actualizar su Q(s, a).

Monte Carlo ES (Exploring Starts), for estimating $\pi \approx \pi_*$

Initialize:
  $\pi(s) \in \mathcal{A}(s)$ (arbitrarily), for all $s \in \mathcal{S}$
  $Q(s,a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$
  $Returns(s,a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Loop forever (for each episode):
  Choose $S_0 \in \mathcal{S}$, $A_0 \in \mathcal{A}(S_0)$ randomly such that all pairs have probability > 0
  Generate an episode from S_0, A_0, following π: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
  $G \leftarrow 0$
  Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
    $G \leftarrow \gamma G + R_{t+1}$
    Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1 \ldots, S_{t-1}, A_{t-1}$:
      Append G to $Returns(S_t, A_t)$
      $Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$
      $\pi(S_t) \leftarrow \text{argmax}_a\, Q(S_t, a)$

CONTROL MONTE CARLO – EJEMPLO

- Entorno: Laberinto 3x3 (A-I).
- Estados Terminales: I (Tesoro, +10), F (Trampa, -5).
- Acciones: UP, DOWN, LEFT, RIGHT (necesarias para ES). Chocar = quedarse.
- Recompensas (R): +10 (entrar I), -5 (entrar F), -0.1 (otros pasos/quedarse).
- Factor de Descuento (γ): 1.

CONTROL MONTE CARLO – EJEMPLO

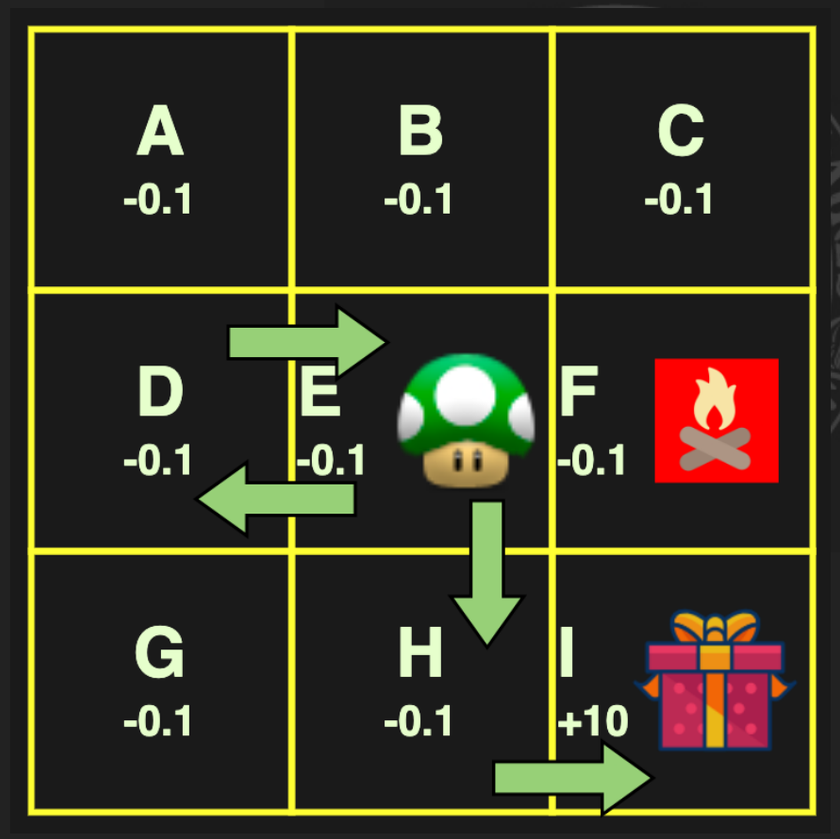
- Inicialización:
- Q(s, a): Inicializa Q(s, a) = 0 para todos los pares estado-acción (s no terminal, a ∈ Acciones).
- Política π(s): Inicializa una política arbitraria (y determinista a la vez):
  - ✓ π(A) = 'RIGHT', π(B) = 'RIGHT', π(C) = 'DOWN'
  - ✓ π(D) = 'RIGHT', π(E) = 'DOWN', # π(F) es terminal
  - ✓ π(G) = 'UP', π(H) = 'RIGHT' # π(I) es terminal
- Returns(s, a): Inicializa listas vacías para acumular los retornos de la primera visita para cada par (s, a).

CONTROL MONTE CARLO – EJEMPLO

- Episodio 1:
- Exploring Start:
  - ✓ Estado inicial al azar: E.
  - ✓ Primera acción al azar: LEFT.

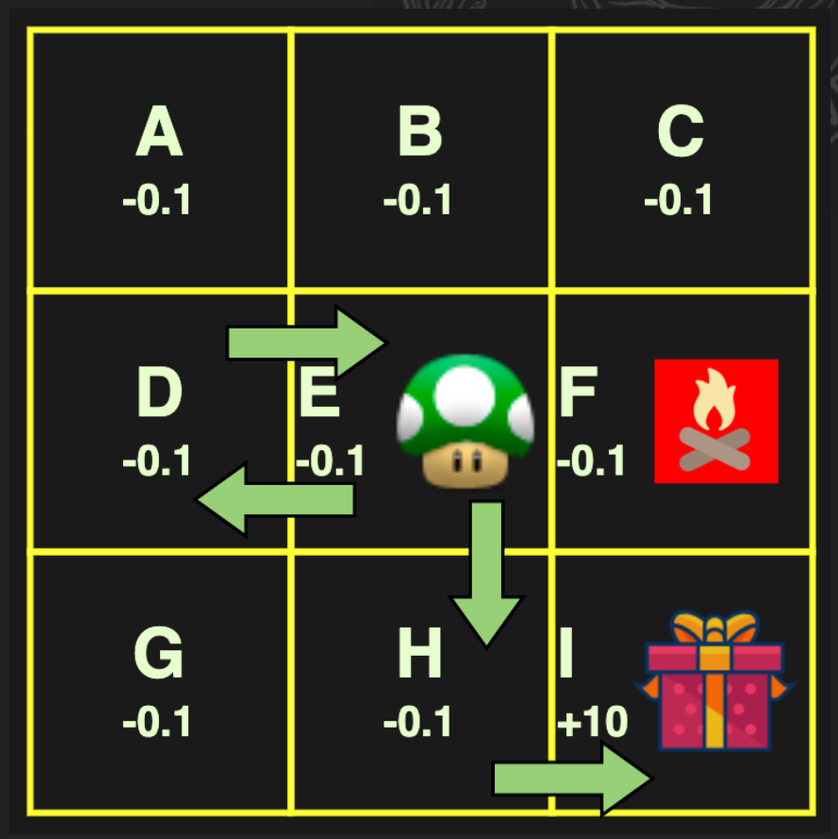
CONTROL MONTE CARLO – EJEMPLO

- Episodio 1:
- Generar Resto del Episodio (siguiendo π inicial):
- Agente en E, ejecuta LEFT. Va a D (R = -0.1).
- Agente en D. Consulta política: π(D) = 'RIGHT'. Ejecuta RIGHT. Va a E (R = -0.1).
- Agente en E. Consulta política: π(E) = 'DOWN'. Ejecuta DOWN. Va a H (R = -0.1).
- Agente en H. Consulta política: π(H) = 'RIGHT'. Ejecuta RIGHT. Va a I (Tesoro!) (R = +10). --> Termina.
- Secuencia: (S_0=E, A_0=LEFT, R_1=-0.1), (S_1=D, A_1=RIGHT, R_2=-0.1), (S_2=E, A_2=DOWN, R_3=-0.1), (S_3=H, A_3=RIGHT, R_4=+10)



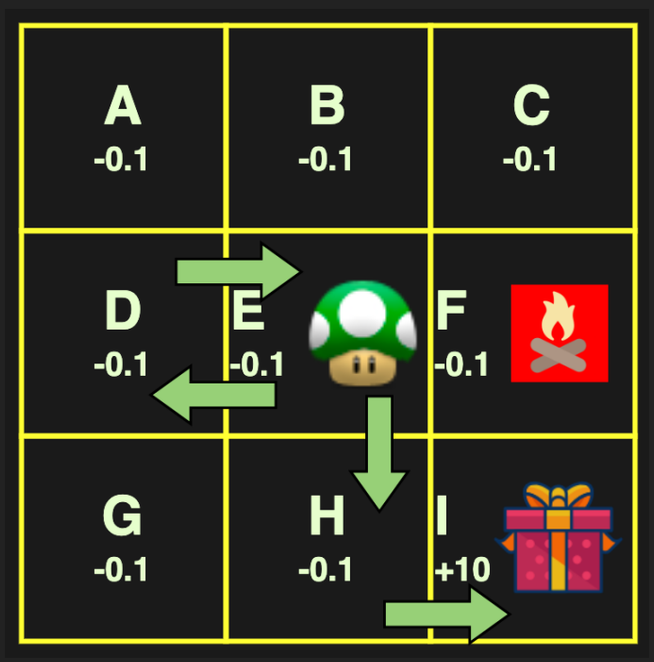
CONTROL MONTE CARLO – EJEMPLO

- Episodio 1:
- Calcular Retornos (G) Hacia Atrás (γ=1):
  - ✓ G para (H, RIGHT) [t=3]: $R_4 + 0 = 10$
  - ✓ G para (E, DOWN) [t=2]: $R_3 + G_3 = -0.1 + 10 = 9.9$
  - ✓ G para (D, RIGHT) [t=1]: $R_2 + G_2 = -0.1 + 9.9 = 9.8$
  - ✓ G para (E, LEFT) [t=0]: $R_1 + G_1 = -0.1 + 9.8 = 9.7$



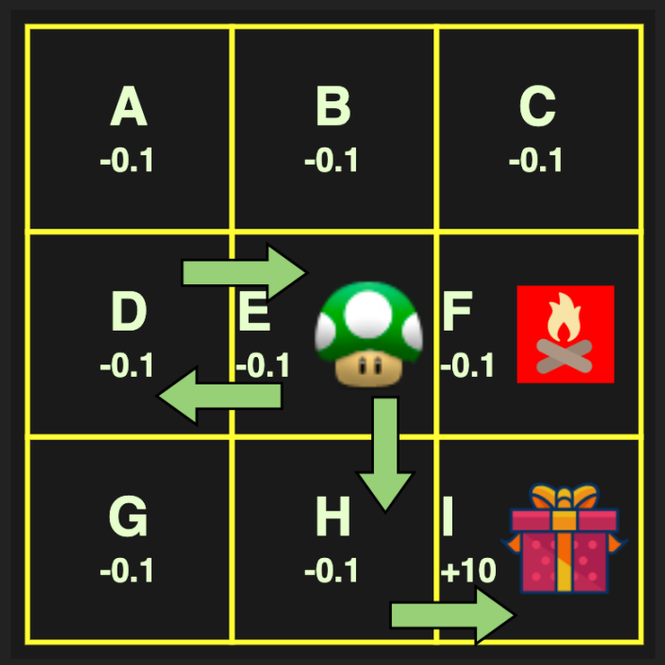
CONTROL MONTE CARLO – EJEMPLO

- Episodio 1:
- Actualizar Q(s, a) (First-Visit del par (s,a)):
  - ✓ Par (H, RIGHT): 1ª visita. Returns[(H, 'RIGHT')].append(10). Q(H, RIGHT) = 10.
  - ✓ Par (E, DOWN): 1ª visita. Returns[(E, 'DOWN')].append(9.9). Q(E, DOWN) = 9.9.
  - ✓ Par (D, RIGHT): 1ª visita. Returns[(D, 'RIGHT')].append(9.8). Q(D, RIGHT) = 9.8.
  - ✓ Par (E, LEFT): 1ª visita. Returns[(E, 'LEFT')].append(9.7). Q(E, LEFT) = 9.7.



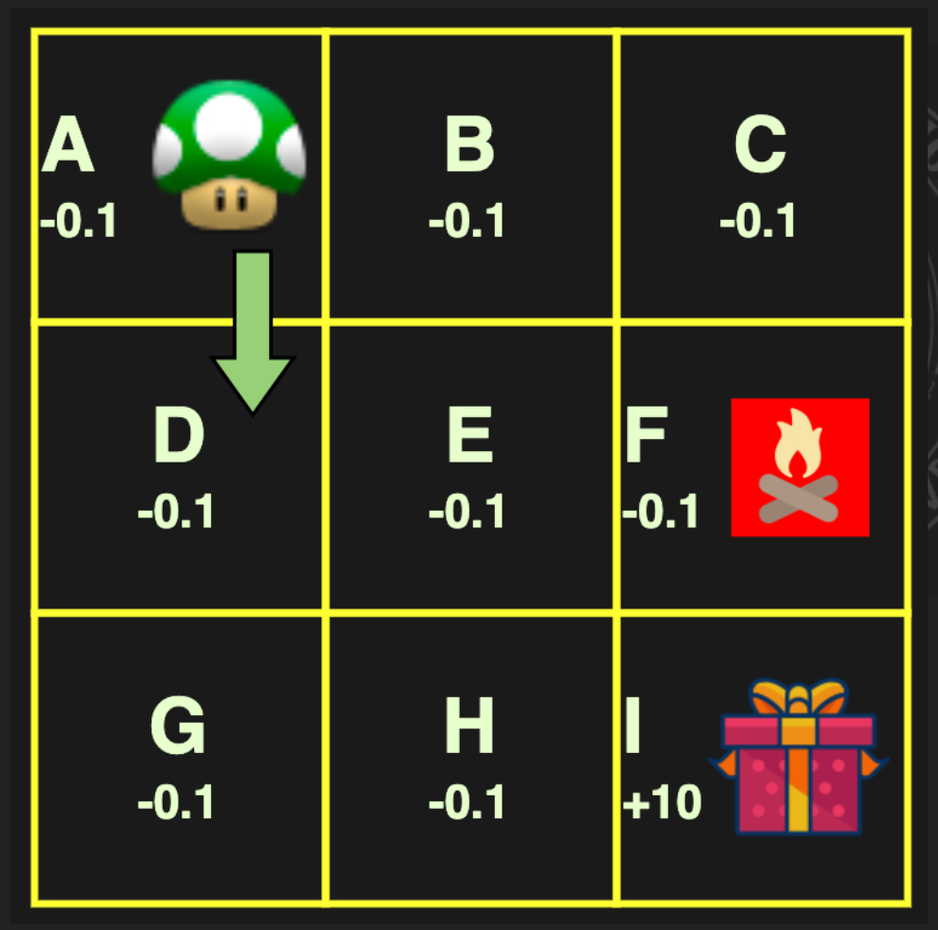
CONTROL MONTE CARLO - EJEMPLO

- Episodio 1:
- Mejorar Política π(s) (Greedy para estados visitados):
  - ✓ Estado H: Q(H, U/D/L)=0, Q(H, R)=10. Mejor='RIGHT'. π(H) = 'RIGHT' (Confirma).
  - ✓ Estado E: Q(E, U/R)=0, Q(E, L)=9.7, Q(E, D)=9.9. Mejor='DOWN'. π(E) = 'DOWN' (Confirma).
  - ✓ Estado D: Q(D, U/D/L)=0, Q(D, R)=9.8. Mejor='RIGHT'. π(D) = 'RIGHT' (Confirma).
  - ✓ La política para A, B, C, G no cambia aún.



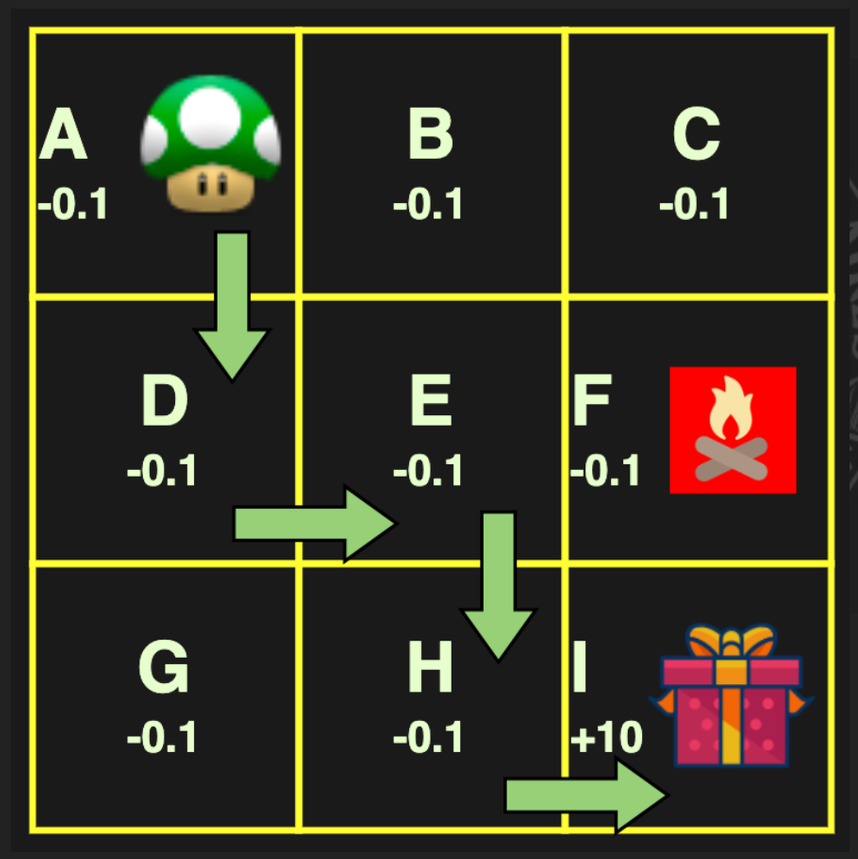
CONTROL MONTE CARLO – EJEMPLO

- Episodio 2:
- Exploring Start:
  - ✓ Estado inicial al azar: A.
  - ✓ Primera acción al azar: DOWN.



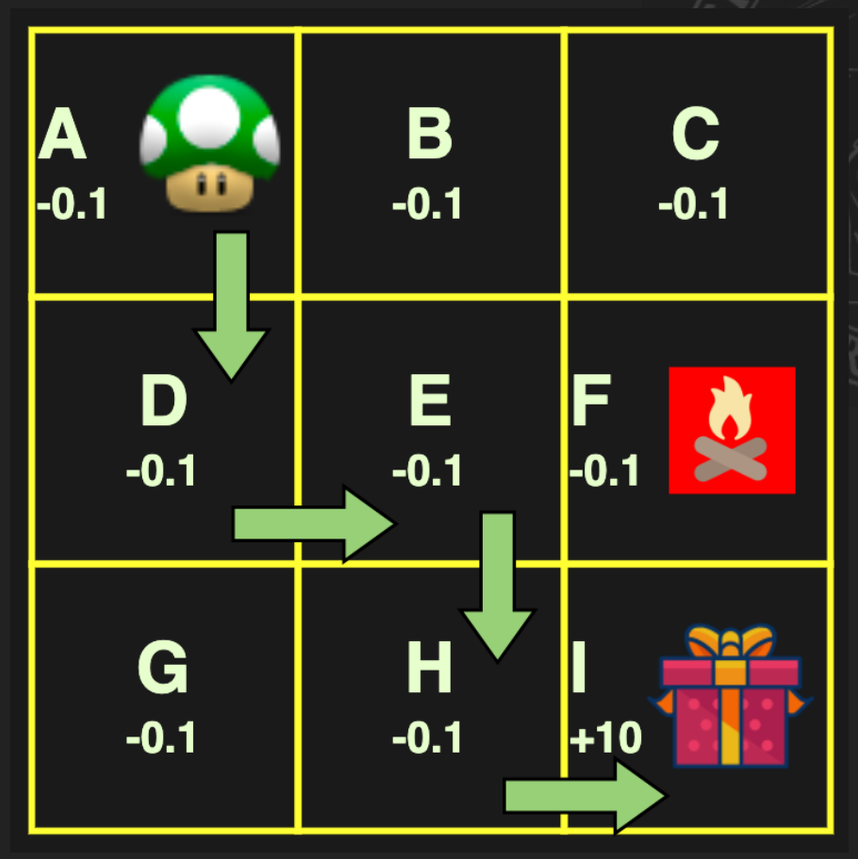
CONTROL MONTE CARLO – EJEMPLO

- Episodio 2:
- Generar Resto del Episodio (siguiendo π actualizada):
- Agente en A, ejecuta DOWN. Va a D (R = -0.1).
- Agente en D. Consulta política: π(D) = 'RIGHT'. Ejecuta RIGHT. Va a E (R = -0.1).
- Agente en E. Consulta política: π(E) = 'DOWN'. Ejecuta DOWN. Va a H (R = -0.1).
- Agente en H. Consulta política: π(H) = 'RIGHT'. Ejecuta RIGHT. Va a I (Tesoro!) (R = +10). --> Termina.
- Secuencia: (S_0=A, A_0=DOWN, R_1=-0.1), (S_1=D, A_1=RIGHT, R_2=-0.1), (S_2=E, A_2=DOWN, R_3=-0.1), (S_3=H, A_3=RIGHT, R_4=+10)



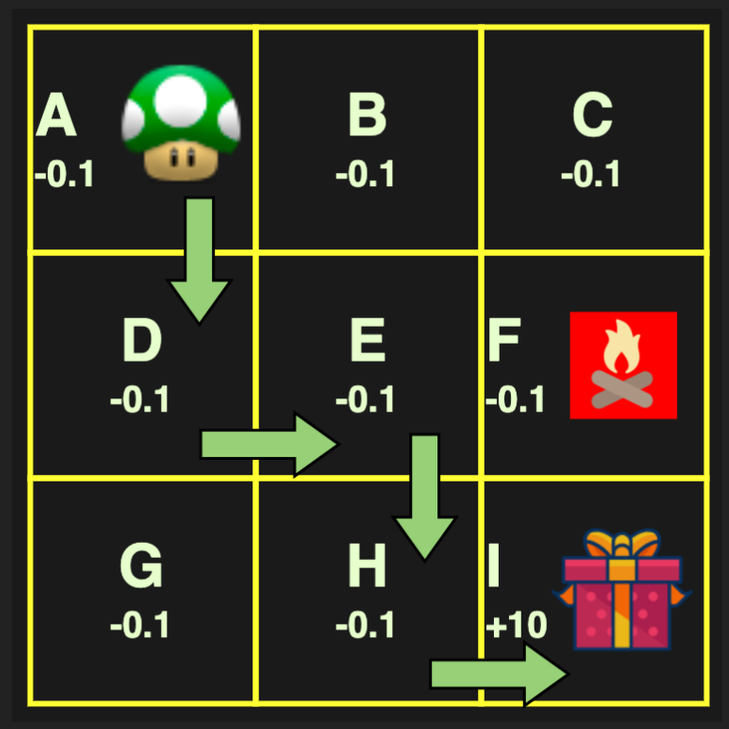
CONTROL MONTE CARLO – EJEMPLO

- Episodio 2:
- Calcular Retornos (G): (Mismos cálculos que en Ep. 1, pero aplicados a estos pares)
  - ✓ G para (H, RIGHT) [t=3]: 10
  - ✓ G para (E, DOWN) [t=2]: 9.9
  - ✓ G para (D, RIGHT) [t=1]: 9.8
  - ✓ G para (A, DOWN) [t=0]: 9.7



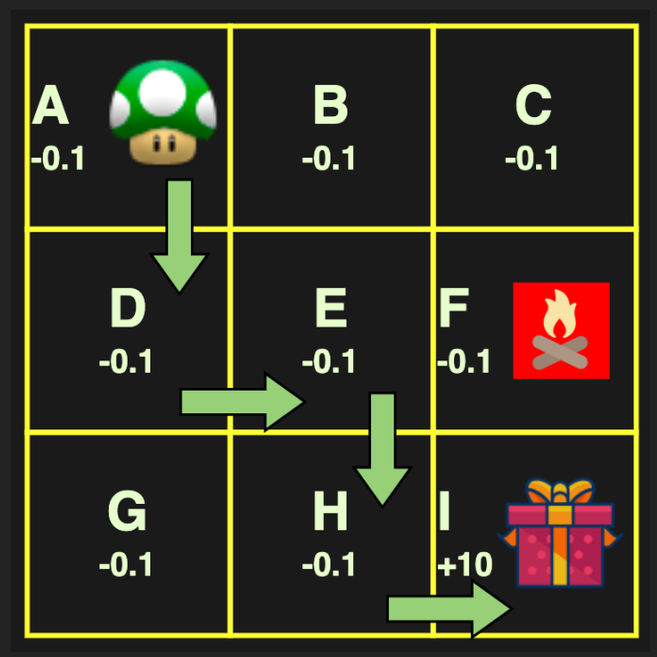
CONTROL MONTE CARLO – EJEMPLO

- Episodio 2:
- Actualizar Q(s, a) (First-Visit del par):
  - ✓ Par (H, RIGHT): 1ª visita en este episodio. Returns[(H, 'RIGHT')] era [10], ahora [10, 10]. Q(H, RIGHT) = 10.
  - ✓ Par (E, DOWN): 1ª visita en este episodio. Returns[(E, 'DOWN')] era [9.9], ahora [9.9, 9.9]. Q(E, DOWN) = 9.9.
  - ✓ Par (D, RIGHT): 1ª visita en este episodio. Returns[(D, 'RIGHT')] era [9.8], ahora [9.8, 9.8]. Q(D, RIGHT) = 9.8.
  - ✓ Par (A, DOWN): 1ª visita. Returns[(A, 'DOWN')].append(9.7). Q(A, DOWN) = 9.7.

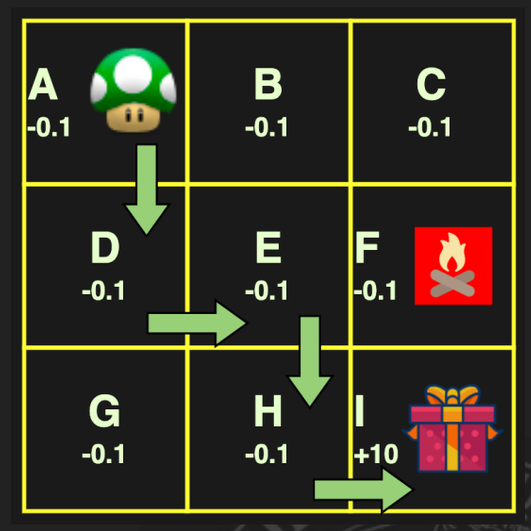


CONTROL MONTE CARLO – EJEMPLO

- Episodio 2:
- Mejorar Política π(s):
  - ✓ Estado H: Q(H, R)=10 (máx). π(H) = 'RIGHT'.
  - ✓ Estado E: Q(E, D)=9.9 (máx). π(E) = 'DOWN'.
  - ✓ Estado D: Q(D, R)=9.8 (máx). π(D) = 'RIGHT'.
  - ✓ Estado A: Q(A, D)=9.7 (máx). π(A) = 'DOWN'. (Cambió).



CONTROL MONTE CARLO – EJEMPLO



- Continuación…
- Se sigue este proceso. Es decir, con cada episodio:
  - ✓ Se explora un nuevo par (s, a) al inicio.
  - ✓ Se sigue la política actual (que es greedy respecto a los Q actuales).
  - ✓ Se calculan los retornos G.
  - ✓ Se actualiza el valor Q del primer par (s',a') encontrado en el episodio que coincide con uno ya visto, usando G.
  - ✓ Se actualiza la política para los estados visitados, haciéndola greedy respecto a los Q recién actualizados.
- Eventualmente, Q(s,a) se acerca a Q*(s,a) y π(s) se acerca a π*(s).
- Los valores de acciones que llevan al tesoro (como Q(H, RIGHT)) serán altos, y los que llevan a la trampa (como Q(C, DOWN), Q(E, RIGHT)) serán bajos.

REFERENCIAS BIBLIOGRÁFICAS Y WEB (I)

- R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.